



Instructor	Mr. Shehryar Raza
Lab Instructor	Muzamil Ali
Semester	Fall 2024

Department of Computer Science ITU, Lahore
Pakistan

Instructions

Please submit a single cpp file titled as “Bscs23xxx_Lab02.cpp”

Part 01

Queue

Task 01: Check for identical queue

Check if two queues are identical (contain the same elements in the same order).

Input: Queue1: 1 2 3 Queue2: 1 2 3	Output: True
---	------------------------

Task 02: Implement Deque (Double-Ended Queue)

You are required to implement a data structure called Deque (Double-Ended Queue). A Deque is a linear data structure that allows insertion and deletion of elements from both ends (front and rear).

You need to implement the following Operations:

insertFront(x): Insert an element x at the front of the deque.

insertRear(x): Insert an element x at the rear of the deque.

deleteFront(): Remove and return the element from the front of the deque. If the deque is empty, return an error message or appropriate indication.

deleteRear(): Remove and return the element from the rear of the deque. If the deque is empty, return an error message or appropriate indication.

getFront(): Get the front element of the deque without removing it. If the deque is empty, return an error message or appropriate indication.

getRear(): Get the rear element of the deque without removing it. If the deque is empty, return an error message or appropriate indication.

isEmpty(): Check if the deque is empty. Return true if empty, otherwise false.

Stack

Task 01. Next Greater Element

Given an array of integers, print the Next Greater Element (NGE) for every element. The Next Greater Element for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exists, print -1.

Input: Arr = { 4, 5, 2, 25 }	Output: { 5, 25, 25, -1 }
--	-------------------------------------

Part 02

Queue

Task 01: Number of Recent Calls

You have a RecentCounter class which counts the number of recent requests within a certain time frame.

Implement the RecentCounter class:

- RecentCounter() Initializes the counter with zero recent requests.
- int ping(int t) Adds a new request at time t, where t represents some time in milliseconds, and returns the number of requests that has happened in the past 3000 milliseconds (including the new request). Specifically, return the number of requests that have happened in the inclusive range [t - 3000, t].
- It is guaranteed that every call to ping uses a strictly larger value of t than the previous call.

Example 1:

Input

```
["RecentCounter", "ping", "ping", "ping", "ping"]  
[[], [1], [100], [3001], [3002]]
```

Output

```
[null, 1, 2, 3, 3]
```

Explanation

```
RecentCounter recentCounter = new RecentCounter();  
recentCounter.ping(1);      // requests = [1], range is [-2999,1], return 1  
recentCounter.ping(100);    // requests = [1, 100], range is [-2900,100], return 2  
recentCounter.ping(3001);   // requests = [1, 100, 3001], range is [1,3001], return 3  
recentCounter.ping(3002);   // requests = [1, 100, 3001, 3002], range is [2,3002], return 3
```

Task 02: Time Needed to Buy Tickets

There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the (n - 1)th person is at the back of the line.

You are given a 0-indexed integer array tickets of length n where the number of tickets that the ith person would like to buy is tickets[i].

Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

Return the time taken for the person at position k (0-indexed) to finish buying tickets.

Example 1:

Input: tickets = [2,3,2], k = 2

Output: 6

Explanation:

- In the first pass, everyone in the line buys a ticket and the line becomes [1, 2, 1].
 - In the second pass, everyone in the line buys a ticket and the line becomes [0, 1, 0].
- The person at position 2 has successfully bought 2 tickets and it took $3 + 3 = 6$ seconds.

Example 2:

Input: tickets = [5,1,1,1], k = 0

Output: 8

Explanation:

- In the first pass, everyone in the line buys a ticket and the line becomes [4, 0, 0, 0].
 - In the next 4 passes, only the person in position 0 is buying tickets.
- The person at position 0 has successfully bought 5 tickets and it took $4 + 1 + 1 + 1 + 1 = 8$ seconds.

Task 03: Find the Winner of the Circular Game

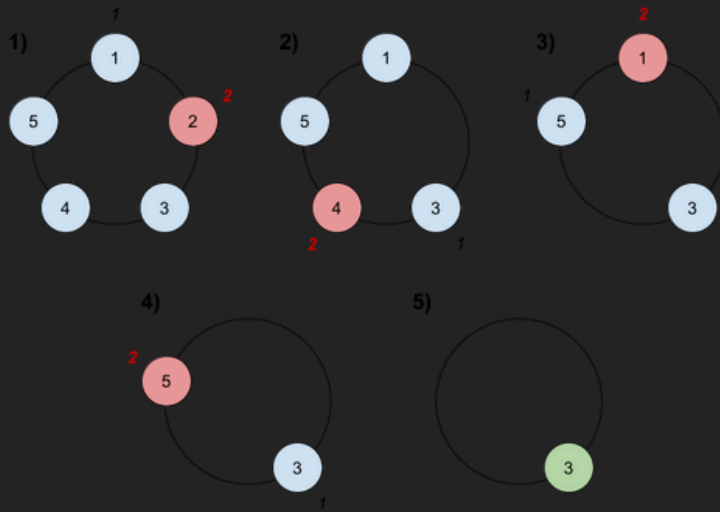
There are n friends that are playing a game. The friends are sitting in a circle and are numbered from 1 to n in clockwise order. More formally, moving clockwise from the i^{th} friend brings you to the $(i+1)^{\text{th}}$ friend for $1 \leq i < n$, and moving clockwise from the n^{th} friend brings you to the 1^{st} friend.

The rules of the game are as follows:

- Start at the 1^{st} friend.
- Count the next k friends in the clockwise direction including the friend you started at. The counting wraps around the circle and may count some friends more than once.
- The last friend you counted leaves the circle and loses the game.
- If there is still more than one friend in the circle, go back to step 2 starting from the friend immediately clockwise of the friend who just lost and repeat.
- Else, the last friend in the circle wins the game.

Given the number of friends, n , and an integer k , return the winner of the game.

Example 1:



Input: $n = 5, k = 2$

Output: 3

Explanation: Here are the steps of the game:

- 1) Start at friend 1.
- 2) Count 2 friends clockwise, which are friends 1 and 2.
- 3) Friend 2 leaves the circle. Next start is friend 3.
- 4) Count 2 friends clockwise, which are friends 3 and 4.
- 5) Friend 4 leaves the circle. Next start is friend 5.
- 6) Count 2 friends clockwise, which are friends 5 and 1.
- 7) Friend 1 leaves the circle. Next start is friend 3.
- 8) Count 2 friends clockwise, which are friends 3 and 5.
- 9) Friend 5 leaves the circle. Only friend 3 is left, so they are the winner.

Example 2:

Input: $n = 6, k = 5$

Output: 1

Explanation: The friends leave in this order: 5, 4, 6, 2, 3. The winner is friend 1.

Stack

Task 01: Number of Students Unable to Eat Lunch

The school cafeteria offers circular and square sandwiches at lunch break, referred to by numbers 0 and 1 respectively. All students stand in a queue. Each student either prefers square or circular sandwiches.

The number of sandwiches in the cafeteria is equal to the number of students. The sandwiches are placed in a stack. At each step:

If the student at the front of the queue prefers the sandwich on the top of the stack, they will take it and leave the queue.

Otherwise, they will leave it and go to the queue's end.

This continues until none of the queue students want to take the top sandwich and are thus unable to eat.

You are given two integer arrays `students` and `sandwiches` where `sandwiches[i]` is the type of the *i*th sandwich in the stack (*i* = 0 is the top of the stack) and `students[j]` is the preference of the *j*th student in the initial queue (*j* = 0 is the front of the queue). Return the number of students that are unable to eat.

Example 1:

Input: `students = [1,1,0,0]`, `sandwiches = [0,1,0,1]`

Output: 0

Explanation:

- Front student leaves the top sandwich and returns to the end of the line making `students = [1,0,0,1]`.
 - Front student leaves the top sandwich and returns to the end of the line making `students = [0,0,1,1]`.
 - Front student takes the top sandwich and leaves the line making `students = [0,1,1]` and `sandwiches = [1,0,1]`.
 - Front student leaves the top sandwich and returns to the end of the line making `students = [1,1,0]`.
 - Front student takes the top sandwich and leaves the line making `students = [1,0]` and `sandwiches = [0,1]`.
 - Front student leaves the top sandwich and returns to the end of the line making `students = [0,1]`.
 - Front student takes the top sandwich and leaves the line making `students = [1]` and `sandwiches = [1]`.
 - Front student takes the top sandwich and leaves the line making `students = []` and `sandwiches = []`.
- Hence all students are able to eat.

Example 2:

Input: `students = [1,1,1,0,0,1]`, `sandwiches = [1,0,0,0,1,1]`

Output: 3

Task 02: Clear Digits

You are given a string `s`.

Your task is to remove all digits by doing this operation repeatedly:

Delete the first digit and the closest non-digit character to its left.

Return the resulting string after removing all digits.

Example 1:

Input: `s = "abc"`

Output: `"abc"`

Explanation:

There is no digit in the string.

Example 2:

Input: `s = "cb34"`

Output: `""`

Explanation:

First, we apply the operation on `s[2]`, and `s` becomes `"c4"`.

Then we apply the operation on `s[1]`, and `s` becomes `""`.